

Python Programming — Complete Concept Notes

Core Language, OOP, and the Small Concepts That Matter

Prepared for Abbas

July 2026

Contents

1	1. Core Syntax & Data Types	3
1.1	1.1 Variables and Dynamic Typing	3
1.2	1.2 Numbers, Strings, Booleans	3
1.3	1.3 Operators	3
1.4	1.4 String Formatting	4
1.5	1.5 Input/Output	4
2	2. Data Structures	5
2.1	2.1 Lists	5
2.2	2.2 Tuples	5
2.3	2.3 Dictionaries	5
2.4	2.4 Sets	6
2.5	2.5 Nested Structures	6
3	3. Control Flow	7
3.1	3.1 if / elif / else and Ternary	7
3.2	3.2 Loops, and the Loop-Else Clause	7
3.3	3.3 range(), enumerate(), zip()	7
4	4. Functions	8
4.1	4.1 Default Args, *args, **kwargs	8
4.2	4.2 Positional-Only and Keyword-Only Arguments	8
4.3	4.3 Lambda Functions	8
4.4	4.4 Scope: Local, Global, Nonlocal, Closures	8
4.5	4.5 Decorators	9
4.6	4.6 Generators	9
5	5. OOP — Core Concepts	11
5.1	5.1 Classes, Objects, __init__, self	11
5.2	5.2 Instance Variables vs. Class Variables	11
5.3	5.3 Instance Methods vs. @classmethod vs. @staticmethod	11
5.4	5.4 Encapsulation	12
5.5	5.5 Inheritance	12
5.6	5.6 Polymorphism	13

5.7	5.7 Abstraction (ABCs)	13
6	6. OOP — Advanced (Often Skipped) Concepts	14
6.1	6.1 Dunder / Magic Methods	14
6.2	6.2 super() and Method Resolution Order (MRO)	14
6.3	6.3 Property Decorators	15
6.4	6.4 Composition vs. Inheritance	15
6.5	6.5 Dataclasses	16
6.6	6.6 __slots__	16
7	7. Error Handling	17
7.1	7.1 try / except / else / finally	17
7.2	7.2 Custom Exceptions	17
7.3	7.3 Context Managers	17
8	8. Iterators & Functional Tools	19
8.1	8.1 Iterator Protocol	19
8.2	8.2 map, filter, reduce	19
8.3	8.3 itertools Basics	19
9	9. Modules & Packaging	21
9.1	9.1 Imports	21
9.2	9.2 Virtual Environments & Dependency Files	21
9.3	9.3 if __name__ == "__main__":	21
10	10. Typing	22
10.1	10.1 Type Hints	22
10.2	10.2 Pydantic Models	22
11	11. Concurrency	23
11.1	11.1 Threading vs. Multiprocessing (Conceptual)	23
11.2	11.2 asyncio	23
12	12. Small Concepts That Trip People Up	25
12.1	12.1 Mutable Default Arguments	25
12.2	12.2 Shallow Copy vs. Deep Copy	25
12.3	12.3 Truthy/Falsy Values, is vs ==	25
12.4	12.4 Late-Binding Closures in Loops	26
12.5	12.5 Pass by “Object Reference”	26
12.6	12.6 Chained Comparisons	27
12.7	12.7 Unpacking with * in Assignments	27
13	Quick-Reference Priority Guide	28

1. Core Syntax & Data Types

1.1 Variables and Dynamic Typing

Python variables are just labels pointing to objects in memory — you don't declare a type, the interpreter infers it at runtime, and the same name can point to different types over its lifetime.

```
x = 10          # x points to an int object
x = "ten"       # now x points to a str object
x = [1, 2, 3]   # now x points to a list object
```

Real-world association: This is exactly why your FastAPI/Pydantic request handlers feel flexible during prototyping but need explicit type hints in production — dynamic typing is fast to write, but in an agent pipeline where a function might unexpectedly receive a `None` instead of a `str` from an LLM response, that flexibility becomes a debugging trap. This is the core reason typed Python (Pydantic models) exists.

1.2 Numbers, Strings, Booleans

- **int** — arbitrary precision (no overflow like in C/Java)
- **float** — IEEE 754 double precision (watch for rounding errors)
- **complex** — $3 + 4j$, rarely used outside scientific computing
- **str** — immutable sequence of Unicode characters
- **bool** — `True/False`, technically a subclass of `int` (`True == 1`)

```
price = 19.99      # float
quantity = 3       # int
name = "GPU Server" # str
in_stock = True    # bool
total = price * quantity # 59.97
```

Real-world association: Floating point rounding errors ($0.1 + 0.2 \neq 0.3$) matter directly when you're computing similarity scores or normalizing embeddings in a RAG pipeline — small float drift can silently change ranking order.

1.3 Operators

- **Arithmetic:** `+` `-` `*` `/` `//` `%` `**`
- **Comparison:** `==` `!=` `>` `<` `>=` `<=`
- **Logical:** `and` `or` `not`
- **Bitwise:** `&` `|` `^` `~` `<<` `>>`
- **Walrus (`:=`):** assign inside an expression (Python 3.8+)

```
# Walrus operator – assign and use in one line
if (n := len(retrieved_docs)) > 5:
    print(f"Retrieved {n} documents, truncating to top 5")
```

Real-world association: The walrus operator shows up constantly in streaming/agent loops where you want to check a condition and reuse the computed value

without a separate line — e.g. `while (chunk := stream.read()):`.

1.4 1.4 String Formatting

```
name, score = "Abbas", 92.5

# f-strings (preferred, Python 3.6+)
print(f"{name} scored {score:.1f}%")

# .format()
print("{} scored {:.1f}%".format(name, score))

# % formatting (legacy, still seen in older codebases)
print("%s scored %.1f%%" % (name, score))
```

Real-world association: f-strings are what you’ll use to build prompts dynamically for LLM calls — inserting retrieved context, user queries, and system instructions into a template string cleanly.

1.5 1.5 Input/Output

```
user_input = input("Enter your query: ")
print("Processing:", user_input)
```

Real-world association: Mostly relevant for CLI tools and quick scripts; in your actual agentic systems, “input” usually comes from an API request body (via FastAPI) rather than `input()`, but it’s the same underlying concept of receiving external data.

2. Data Structures

2.1 Lists

Ordered, mutable, allow duplicates.

```
docs = ["intro.pdf", "chapter1.pdf", "chapter2.pdf"]
docs.append("appendix.pdf")

# Slicing
first_two = docs[:2]

# List comprehension
pdf_names = [d.upper() for d in docs if d.endswith(".pdf")]
```

Real-world association: Lists are what you'll use to hold a batch of retrieved chunks, a conversation history, or a sequence of agent steps — anything ordered that can grow.

2.2 Tuples

Ordered, **immutable**, allow duplicates.

```
point = (35.7, 139.7) # e.g. (latitude, longitude)
lat, lon = point      # unpacking

# Star unpacking
first, *rest = (1, 2, 3, 4)
```

Real-world association: Tuples are ideal for fixed-shape data you don't want accidentally mutated — like returning (embedding_vector, metadata) from a function, where the shape is guaranteed and shouldn't change downstream.

2.3 Dictionaries

Key-value pairs, insertion-ordered (Python 3.7+), mutable.

```
agent_config = {"model": "claude", "temperature": 0.7}

# Safe access
timeout = agent_config.get("timeout", 30) # default if missing

# Dict comprehension
scores = {"a": 90, "b": 75, "c": 60}
passed = {k: v for k, v in scores.items() if v >= 70}

from collections import defaultdict, Counter
graph = defaultdict(list) # auto-creates empty list for new keys
word_freq = Counter("agentic ai agentic".split())
```

Real-world association: This is your bread and butter for JSON-like structures —

API payloads, agent state objects, tool call arguments — basically every FastAPI request/response body is dict-shaped under the hood.

2.4 2.4 Sets

Unordered, mutable, **no duplicates**.

```
seen_urls = {"a.com", "b.com"}
seen_urls.add("c.com")

new_urls = {"b.com", "c.com", "d.com"}
to_crawl = new_urls - seen_urls      # difference: {"d.com"}
overlap = new_urls & seen_urls       # intersection
combined = new_urls | seen_urls      # union
```

Real-world association: Sets are perfect for deduplication — e.g. deduplicating retrieved document IDs across multiple retriever calls in a RAG pipeline before passing context to the LLM.

2.5 2.5 Nested Structures

```
agent_state = {
    "messages": [{"role": "user", "content": "hi"}],
    "tools_used": ["search", "calculator"],
    "metadata": {"session_id": "abc123", "retries": 0}
}
```

Real-world association: This is literally the shape of a LangGraph state object — nested dicts/lists are how you'll represent conversation state, tool outputs, and memory in every agent you build.

3. Control Flow

3.1 if / elif / else and Ternary

```
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"

# Ternary (conditional expression)
status = "pass" if score >= 60 else "fail"
```

Real-world association: Ternaries are common in one-line config defaults, e.g. `model = "gpt-4" if use_premium else "gpt-3.5"`.

3.2 Loops, and the Loop-Else Clause

```
for doc in docs:
    if doc.corrupted:
        break
else:
    print("All documents processed cleanly") # runs only if no break occurred
```

The `else` on a loop is one of Python's most-missed small concepts: it runs **only if the loop completed without hitting `break`**.

Real-world association: Useful in validation loops — e.g. checking every retrieved chunk passes a relevance threshold, and only proceeding to generation if none failed.

3.3 range(), enumerate(), zip()

```
for i in range(5):                # 0..4
    pass

for i, doc in enumerate(docs):    # index + value
    print(i, doc)

names = ["a", "b"]
scores = [90, 75]
for name, score in zip(names, scores): # pair up two sequences
    print(name, score)
```

Real-world association: `zip()` is exactly how you'd pair a list of documents with their similarity scores when ranking retrieval results.

4 4. Functions

4.1 4.1 Default Args, *args, **kwargs

```
def call_llm(prompt, temperature=0.7, *args, **kwargs):  
    print(prompt, temperature, args, kwargs)
```

```
call_llm("Hello", 0.9, "extra_positional", model="claude", max_tokens=500)
```

*args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict.

Real-world association: This pattern is everywhere in wrapper functions around LLM SDKs — you accept a core set of named params but forward arbitrary extra options (**kwargs) straight to the underlying API call.

4.2 4.2 Positional-Only and Keyword-Only Arguments

```
def connect(host, port, /, *, timeout=30):  
    # host, port: positional-only (before /)  
    # timeout: keyword-only (after *)  
    ...
```

```
connect("localhost", 8000, timeout=10)    # valid  
connect(host="localhost", port=8000)      # invalid – host/port must be positional
```

Real-world association: Used in library/API design to lock down a function signature so callers can't accidentally pass things by the wrong name — common in well-designed SDKs like FastAPI's own internals.

4.3 4.3 Lambda Functions

```
docs.sort(key=lambda d: d["score"], reverse=True)
```

Anonymous, single-expression functions — used inline where a full def would be overkill.

Real-world association: You'll use lambdas constantly as the key= argument when sorting retrieved documents by relevance score.

4.4 4.4 Scope: Local, Global, Nonlocal, Closures

```
def make_counter():  
    count = 0  
    def increment():  
        nonlocal count    # modifies the enclosing function's variable  
        count += 1  
    return count
```



```

    return increment

counter = make_counter()
counter() # 1
counter() # 2

```

A **closure** is a function that “remembers” variables from its enclosing scope even after that scope has finished executing.

Real-world association: Closures are the mechanism behind decorators and behind callback functions you pass into agent frameworks — e.g. a tool function that closes over a database connection or API client.

4.5 4.5 Decorators

```

import time
from functools import wraps

def log_latency(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.2f}s")
        return result
    return wrapper

@log_latency
def call_model(prompt):
    ...

```

A decorator wraps a function to add behavior without modifying its internals.

Real-world association: This is a core pattern in your stack — FastAPI route decorators (`@app.post(...)`), LangChain/LangGraph tool decorators (`@tool`), and your own logging/retry decorators around LLM API calls all use exactly this mechanism.

4.6 4.6 Generators

```

def stream_tokens(text, chunk_size=4):
    for i in range(0, len(text), chunk_size):
        yield text[i:i + chunk_size]

for token in stream_tokens("Hello world"):
    print(token)

```

`yield` pauses the function and returns a value, resuming from that point on the next call — generators produce values lazily, one at a time, instead of building a full list in

memory.

Real-world association: This is exactly how LLM streaming responses work under the hood — tokens are yielded one at a time as they arrive, rather than waiting for the full response before returning anything.

5. OOP — Core Concepts

5.1 5.1 Classes, Objects, __init__, self

```
class Agent:
    def __init__(self, name, model):
        self.name = name
        self.model = model

    def respond(self, prompt):
        return f"{self.name} using {self.model}: {prompt}"

research_agent = Agent("Researcher", "claude-sonnet")
print(research_agent.respond("Find papers on GNNs"))
```

`self` refers to the specific instance; `__init__` is the constructor, run automatically when a new object is created.

Real-world association: Every agent class in a LangGraph/agentic system is structured this way — a class holding config (model, tools, memory) with methods that act on that state.

5.2 5.2 Instance Variables vs. Class Variables

```
class Agent:
    total_agents = 0          # class variable – shared across all instances

    def __init__(self, name):
        self.name = name      # instance variable – unique per object
        Agent.total_agents += 1

a1 = Agent("A")
a2 = Agent("B")
print(Agent.total_agents)    # 2
```

Real-world association: Class variables are useful for shared config or counters — e.g. tracking total API calls made across all agent instances in a session.

5.3 5.3 Instance Methods vs. @classmethod vs. @staticmethod

```
class Agent:
    def __init__(self, name):
        self.name = name

    def greet(self):           # instance method – needs self
        return f"I am {self.name}"
```

```

@classmethod
def from_config(cls, config):                # classmethod – needs cls, not self
    return cls(config["name"])

@staticmethod
def is_valid_name(name):                    # staticmethod – needs neither
    return len(name) > 0

agent = Agent.from_config({"name": "Bot"})

```

Real-world association: @classmethod is exactly how you’d build a from_json or from_config constructor for loading an agent’s settings from a config file — very common in real agent frameworks.

5.4 5.4 Encapsulation

```

class Wallet:
    def __init__(self, balance):
        self._balance = balance            # protected (convention only)
        self.__pin = "1234"               # name-mangled to _Wallet__pin

    def get_balance(self):
        return self._balance

```

Python doesn’t enforce true private access — `_` is a convention (“don’t touch this”), `__` triggers name mangling to discourage accidental access from subclasses.

Real-world association: You’ll use `_` prefixes to mark internal helper methods in your agent classes that aren’t part of the public API you expose to other developers.

5.5 5.5 Inheritance

```

class BaseAgent:
    def __init__(self, name):
        self.name = name

    def act(self):
        raise NotImplementedError

class SearchAgent(BaseAgent):
    def act(self):
        return f"{self.name} is searching the web"

```

Real-world association: This is exactly how LangChain/LangGraph tool and agent base classes work — you inherit from a BaseTool or BaseAgent and override the required methods.

5.6 5.6 Polymorphism

```
def run_agent(agent):  
    print(agent.act())    # works for ANY object with an .act() method  
  
run_agent(SearchAgent("Bot1"))  
run_agent(CalculatorAgent("Bot2"))
```

Polymorphism (including **duck typing** — “if it walks like a duck and quacks like a duck”) means code can work with different types as long as they support the same interface.

Real-world association: This is why you can swap out one tool or one LLM provider for another in an agent pipeline without rewriting the orchestration logic — as long as both implement the same `.invoke()` method.

5.7 5.7 Abstraction (ABCs)

```
from abc import ABC, abstractmethod  
  
class BaseTool(ABC):  
    @abstractmethod  
    def run(self, query):  
        ...  
  
class SearchTool(BaseTool):  
    def run(self, query):  
        return f"Searching for {query}"  
  
# BaseTool() -> TypeError: can't instantiate abstract class
```

An Abstract Base Class defines a required interface — subclasses **must** implement the abstract methods or Python raises an error.

Real-world association: This is exactly the pattern behind `BaseTool` in `LangChain` — it forces every custom tool you write to implement a `.run()/.invoke()` method, guaranteeing your agent orchestration code can call any tool the same way.

6 6. OOP — Advanced (Often Skipped) Concepts

6.1 6.1 Dunder / Magic Methods

```
class Document:
    def __init__(self, title, length):
        self.title = title
        self.length = length

    def __str__(self):
        return f"Document: {self.title}"           # for print()

    def __repr__(self):
        return f"Document(title={self.title!r})"   # for debugging/repr

    def __eq__(self, other):
        return self.length == other.length

    def __len__(self):
        return self.length

    def __iter__(self):
        return iter(range(self.length))

d = Document("Report", 10)
print(d)           # uses __str__
print(len(d))      # uses __len__
```

Real-world association: Implementing `__eq__` and `__len__` on a custom `Document` or `Chunk` class lets you use built-in Python operations (`==`, `len()`, `sorted()`) naturally on your RAG pipeline's data objects instead of writing custom comparison functions everywhere.

6.2 6.2 super() and Method Resolution Order (MRO)

```
class BaseAgent:
    def __init__(self, name):
        self.name = name
        print("BaseAgent init")

class LoggingAgent(BaseAgent):
    def __init__(self, name, log_level):
        super().__init__(name)           # calls BaseAgent's __init__
        self.log_level = log_level
        print("LoggingAgent init")

agent = LoggingAgent("Bot", "DEBUG")
```

```
print(LoggingAgent.__mro__)  # shows the resolution order for multiple inheritance
```

`super()` calls the next class in the MRO chain — essential when a subclass needs to extend, not replace, the parent's behavior.

Real-world association: Any time you subclass a LangChain BaseTool or a FastAPI base model and add extra setup, you'll call `super().__init__(...)` to make sure the parent's setup still runs.

6.3 6.3 Property Decorators

```
class Agent:
    def __init__(self, temperature):
        self._temperature = temperature

    @property
    def temperature(self):
        return self._temperature

    @temperature.setter
    def temperature(self, value):
        if not 0 <= value <= 2:
            raise ValueError("Temperature must be between 0 and 2")
        self._temperature = value

agent = Agent(0.7)
agent.temperature = 1.5  # runs validation via the setter
```

`@property` lets you access a method like an attribute (`agent.temperature`, no parentheses) while still running validation logic behind the scenes.

Real-world association: This is the Pythonic way to add validation to a config value — like making sure an LLM's temperature or top_p parameter never gets set outside a valid range, without callers needing to call a `.set_temperature()` method explicitly.

6.4 6.4 Composition vs. Inheritance

```
# Inheritance: "is-a"
class SearchAgent(BaseAgent):
    ...

# Composition: "has-a" — usually more flexible
class Agent:
    def __init__(self, tools):
        self.tools = tools  # a list of Tool objects, not inherited behavior

    def run(self, query):
```

```
for tool in self.tools:
    tool.run(query)
```

Real-world association: Modern agent frameworks (LangGraph included) lean heavily toward composition — an agent “has” a list of tools and a memory object, rather than inheriting from a giant class hierarchy. This is more flexible when you want to mix and match tools at runtime.

6.5 6.5 Dataclasses

```
from dataclasses import dataclass, field

@dataclass
class RetrievedChunk:
    text: str
    score: float
    metadata: dict = field(default_factory=dict)

chunk = RetrievedChunk(text="...", score=0.87)
print(chunk)  # auto-generated __repr__: RetrievedChunk(text='...', score=0.87, metadata={})
```

@dataclass auto-generates `__init__`, `__repr__`, and `__eq__` for classes that are mostly just structured data.

Real-world association: You’ll use dataclasses (or their close cousin, Pydantic models) constantly to define the shape of tool inputs/outputs and agent state — much less boilerplate than writing `__init__` by hand for every data-holding class.

6.6 6.6 __slots__

```
class Point:
    __slots__ = ["x", "y"]  # no __dict__ created – saves memory, blocks new attributes

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
p.z = 3  # AttributeError – z is not in __slots__
```

Real-world association: Worth knowing exists mainly for memory optimization — relevant if you’re instantiating millions of small objects (e.g. one object per embedding vector) and want to cut memory overhead.

7 7. Error Handling

7.1 7.1 try / except / else / finally

```
try:
    response = call_llm_api(prompt)
except TimeoutError:
    response = "Request timed out"
except ValueError as e:
    response = f"Invalid input: {e}"
else:
    print("Call succeeded")           # runs only if no exception occurred
finally:
    log_request()                   # always runs, exception or not
```

Real-world association: This is the backbone of resilient agent code — every external API call (LLM, retriever, tool) should be wrapped this way so one failed call doesn't crash the whole pipeline.

7.2 7.2 Custom Exceptions

```
class ToolExecutionError(Exception):
    def __init__(self, tool_name, message):
        self.tool_name = tool_name
        super().__init__(f"{tool_name}: {message}")

raise ToolExecutionError("web_search", "API key missing")
```

Real-world association: Custom exceptions let you distinguish “the LLM API failed” from “the tool failed” from “the retriever failed” in your error handling, so you can retry or fall back differently depending on which part of the pipeline broke.

7.3 7.3 Context Managers

```
class DBSession:
    def __enter__(self):
        print("Opening connection")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Closing connection")

with DBSession() as session:
    ... # connection is guaranteed to close even if an error occurs here

# Function-based version
from contextlib import contextmanager
```

```
@contextmanager
def timer():
    import time
    start = time.time()
    yield
    print(f"Elapsed: {time.time() - start:.2f}s")

with timer():
    call_llm_api(prompt)
```

Real-world association: You'll use `with` constantly for database sessions, file handles, and API client connections — e.g. FastAPI's dependency-injected DB sessions, or wrapping an LLM call to guarantee cleanup/logging regardless of success or failure.

8. Iterators & Functional Tools

8.1 Iterator Protocol

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

for n in Countdown(3):
    print(n)    # 3, 2, 1
```

Any object implementing `__iter__` and `__next__` can be used in a for loop — this is what makes lists, generators, and custom classes all “iterable” under one shared interface.

Real-world association: Understanding this protocol clarifies why generators (Section 4.6) work seamlessly with for loops — a generator is just a shortcut for writing this exact pattern.

8.2 map, filter, reduce

```
from functools import reduce

scores = [70, 85, 90, 60]

doubled = list(map(lambda x: x * 2, scores))
passing = list(filter(lambda x: x >= 70, scores))
total = reduce(lambda acc, x: acc + x, scores, 0)
```

Real-world association: `filter()` is a clean way to drop low-relevance retrieved chunks below a similarity threshold before passing context to the LLM; `map()` is useful for transforming a batch of documents (e.g. running the same embedding function over all of them).

8.3 itertools Basics

```
from itertools import chain, product, groupby
```

```
# chain: flatten multiple iterables into one
combined = list(chain([1, 2], [3, 4]))    # [1, 2, 3, 4]

# product: cartesian product (all combinations)
combos = list(product(["gpt", "claude"], [0.3, 0.7]))

# groupby: group consecutive items by a key
data = [("a", 1), ("a", 2), ("b", 3)]
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, list(group))
```

Real-world association: `product()` is handy for generating hyperparameter sweep combinations (model × temperature) when benchmarking different agent configs; `chain()` is useful for merging results from multiple retrievers into one stream.

9. Modules & Packaging

9.1 Imports

```
import os                                # absolute import
from typing import List, Optional        # import specific names
from . import utils                      # relative import (within a package)
from .tools import search_tool          # relative import from a sibling module
```

`__init__.py` marks a directory as a Python package and can control what's exposed when the package is imported.

Real-world association: Structuring a real agentic project (e.g. separating tools/, agents/, prompts/ into packages) depends entirely on getting this right — a messy import structure is one of the most common sources of friction in growing AI projects.

9.2 Virtual Environments & Dependency Files

```
python -m venv .venv
source .venv/bin/activate      # Linux/Mac
pip install -r requirements.txt
```

`requirements.txt` pins simple dependency lists; `pyproject.toml` is the modern standard, also handling build config and tool settings (e.g. for black, ruff).

Real-world association: Every FastAPI/LangGraph project you ship needs an isolated environment — mixing dependency versions across projects (e.g. two projects needing different LangChain versions) is a very real, very common breakage.

9.3 `if __name__ == "__main__":`

```
def main():
    print("Running agent pipeline")

if __name__ == "__main__":
    main()
```

This block only runs when the file is executed directly — not when it's imported as a module elsewhere.

Real-world association: Lets you write a script that's both a runnable CLI tool (`python run_agent.py`) and an importable module (`from run_agent import main`) without the script accidentally executing on import.

10 10. Typing

10.1 10.1 Type Hints

```
def summarize(text: str, max_length: int = 100) -> str:
    return text[:max_length]

from typing import List, Dict, Optional, Union

def process_docs(docs: List[str], config: Optional[Dict[str, int]] = None) -> Union[str, ...]
```

Type hints don't enforce types at runtime by themselves — they're primarily for readability, editor autocomplete, and static analysis tools (mypy).

Real-world association: Type hints are what make FastAPI's automatic request validation and OpenAPI docs generation work — FastAPI reads your function's type hints to build the entire API schema automatically.

10.2 10.2 Pydantic Models

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    prompt: str
    temperature: float = Field(default=0.7, ge=0, le=2)
    max_tokens: int = 500

# Pydantic actually validates at runtime, unlike plain type hints
request = ChatRequest(prompt="Hello", temperature=3.0) # raises ValidationError
```

Pydantic extends plain typing with **runtime enforcement** — invalid data raises a clear error instead of silently causing a bug three functions downstream.

Real-world association: This is the exact mechanism validating every request body your FastAPI agent endpoints receive, and it's also how structured outputs are enforced when parsing an LLM's JSON response into a typed object.

11 11. Concurrency

11.1 11.1 Threading vs. Multiprocessing (Conceptual)

- **Threading:** multiple threads share the same memory space; good for I/O-bound work (waiting on network calls), limited by Python's GIL for CPU-bound work.
- **Multiprocessing:** separate processes, separate memory; good for CPU-bound work (heavy computation), higher overhead to set up.

```
# Threading – good for many parallel API calls
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=5) as executor:
    results = list(executor.map(call_llm_api, prompts))

# Multiprocessing – good for CPU-heavy work like embedding generation
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor() as executor:
    embeddings = list(executor.map(compute_embedding, documents))
```

Real-world association: Threading is what you'd reach for to fire off multiple LLM API calls in parallel (I/O-bound, waiting on network); multiprocessing is better if you're running local embedding models or heavy numeric preprocessing (CPU-bound).

11.2 11.2 asyncio

```
import asyncio

async def call_llm_async(prompt):
    await asyncio.sleep(1)  # simulates a network call
    return f"Response to: {prompt}"

async def main():
    # Run multiple LLM calls concurrently, not sequentially
    results = await asyncio.gather(
        call_llm_async("prompt 1"),
        call_llm_async("prompt 2"),
        call_llm_async("prompt 3"),
    )
    print(results)

asyncio.run(main())
```

async/await let a single thread juggle many I/O-bound operations concurrently by pausing at each await point instead of blocking.

Real-world association: This is the backbone of every production FastAPI endpoint

and every modern agent framework — `async def` route handlers, async LLM SDK calls, and `asyncio.gather()` for running multiple tool calls concurrently instead of one at a time, which is often the single biggest latency win in an agent pipeline.

12 12. Small Concepts That Trip People Up

12.1 12.1 Mutable Default Arguments

```
# WRONG – the same list is reused across every call
def add_message(msg, history=[]):
    history.append(msg)
    return history

# CORRECT
def add_message(msg, history=None):
    if history is None:
        history = []
    history.append(msg)
    return history
```

Default argument values are evaluated **once**, when the function is defined — not on every call. A mutable default (list, dict) is shared and silently accumulates state across calls.

Real-world association: This exact bug shows up in conversation-history or agent-memory functions — if you default a messages parameter to [], every agent instance that doesn't explicitly pass its own history ends up secretly sharing (and polluting) the same list.

12.2 12.2 Shallow Copy vs. Deep Copy

```
import copy

original = {"tools": ["search", "calc"], "config": {"temp": 0.7}}

shallow = copy.copy(original)           # nested objects are still shared
deep = copy.deepcopy(original)          # fully independent copy

shallow["config"]["temp"] = 1.0
print(original["config"]["temp"])      # 1.0 – changed! shallow copy shared the nested c
```

Real-world association: When you clone an agent's config to run a variant experiment, a shallow copy can silently corrupt your original config's nested values — always deep-copy config dicts before mutating a “variant” version.

12.3 12.3 Truthy/Falsy Values, is vs ==

```
# Falsy: 0, 0.0, "", [], {}, set(), None, False
if not retrieved_docs:           # true if the list is empty
    print("No results found")
```

```
# == checks value equality; `is` checks identity (same object in memory)
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True – same values
print(a is b)    # False – different objects

x = None
print(x is None) # correct way to check for None – never use x == None
```

Real-world association: Checking if response is None vs if not response matters a lot when an LLM call could return an empty string (falsy but not an error) versus an actual failure (None) — conflating the two is a common source of silently swallowed bugs.

12.4 12.4 Late-Binding Closures in Loops

```
# WRONG – all lambdas capture the SAME variable `i`, not its value at creation time
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs]) # [2, 2, 2] – not [0, 1, 2] as you'd expect

# CORRECT – force early binding with a default argument
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs]) # [0, 1, 2]
```

Real-world association: This bites people building a list of callback/tool functions in a loop — e.g. dynamically creating a tool per document — where every callback ends up referencing the *last* document instead of its own.

12.5 12.5 Pass by “Object Reference”

```
def modify_list(lst):
    lst.append(4)          # mutates the original object

def reassign_list(lst):
    lst = [9, 9, 9]       # only rebinds the LOCAL name, original is untouched

nums = [1, 2, 3]
modify_list(nums)
print(nums) # [1, 2, 3, 4] – mutated

reassign_list(nums)
print(nums) # [1, 2, 3, 4] – unchanged, reassignment didn't propagate
```

Python isn’t strictly “pass by value” or “pass by reference” — it passes the object reference by value, so mutating an object inside a function affects the caller, but reassigning the parameter name does not.

Real-world association: This distinction explains a lot of confusing agent-state bugs — mutating a shared state dict inside a tool function affects the whole pipeline, but reassigning the parameter to a new dict silently does nothing to the caller’s state.

12.6 12.6 Chained Comparisons

```
temperature = 0.7
if 0 <= temperature <= 1:      # equivalent to: 0 <= temperature and temperature <= 1
    print("Valid range")
```

Real-world association: Cleaner than writing two separate and-joined conditions when validating that a parameter (temperature, top_p, a similarity score) falls within a valid range.

12.7 12.7 Unpacking with * in Assignments

```
first, *middle, last = [1, 2, 3, 4, 5]
print(first, middle, last)    # 1 [2, 3, 4] 5

# Common real pattern: separate the top result from the rest
top_result, *remaining_results = ranked_documents
```

Real-world association: Handy for cleanly separating the best-ranked retrieval result from the rest of the candidates without slicing by index (results[0], results[1:]), which is more error-prone and less readable.

13 Quick-Reference Priority Guide

Given your current stack — **FastAPI, Pydantic, LangGraph/LangChain, async agent pipelines** — these concepts carry the most direct, everyday weight:

Priority	Concepts
Highest	Decorators, Dataclasses/Pydantic, Generators, Context Managers, asyncio, Type Hints
High	Dictionaries/Sets, Closures, Inheritance + Composition, Custom Exceptions, *args/**kwargs
Medium	Dunder methods, @property, @classmethod, itertools, threading vs multiprocessing
Good to know	<code>__slots__</code> , MRO details, positional/keyword-only args, walrus operator

The “small gotchas” in Section 12 — especially mutable default arguments, shallow vs. deep copy, and closures in loops — are worth memorizing specifically because they cause *silent* bugs in agent state and config handling rather than loud crashes, which makes them disproportionately costly in production agentic systems.